Lecture 13: Advanced Binary Search Applications

1. The Boundary Modification Paradigm

When dealing with duplicate elements or monotonic slopes, standard Binary Search (which halts immediately upon finding a match) is insufficient. We must modify the search space boundaries even *after* a match is found to strictly isolate the mathematical extrema (the absolute first occurrence, last occurrence, or physical peak).

Problem-Solving Deconstruction

Problem 1: First and Last Position of an Element in a Sorted Array

1. Problem Statement & Constraints: Given a strictly sorted array containing duplicate elements and a target `key`, find the exact starting and ending index of that target. If the target is not found, return `[-1, -1]`.

2. Core Intuition: Because the array is sorted, all duplicate target values will be grouped contiguously.

- **First Occurrence:** If we find the `key` at `mid`, it might be the first occurrence, or there might be more occurrences to its left. We record the `mid` index as a potential answer but force the search space to continue left (`end = mid - 1`) to check for earlier matches.
- **Last Occurrence:** Conversely, if we find the `key`, we record it but force the search space to continue right (`start = mid + 1`) to check for subsequent matches.

3. Algorithmic Steps (First Occurrence):

1. Initialize `start = 0`, `end = size - 1`, and a tracker `ans = -1`.
2. Open a `while (start <= end)` loop and calculate the safe `mid`.
3. Evaluate match: `if (arr[mid] == key)`. If True, store `ans = mid` and aggressively update `end = mid - 1` to search the left sub-array.
4. Evaluate right shift: `else if (key > arr[mid])`, update `start = mid + 1`.
5. Evaluate left shift: `else if (key < arr[mid])`, update `end = mid - 1`.
6. Return `ans`.

(Note: The Last Occurrence algorithm is identical, except step 3 updates `start = mid + 1`).

4. Dry Run (First Occurrence):

Input: `arr = [1, 2, 4, 4, 4, 4, 9]`, `key = 4`

1. `start = 0, end = 6`. `mid = 3` (value `4`). `ans = 3`. Update `end = 2`.
2. `start = 0, end = 2`. `mid = 1` (value `2`). `4 > 2`. Update `start = 2`.
3. `start = 2, end = 2`. `mid = 2` (value `4`). `ans = 2`. Update `end = 1`.
4. `start = 2, end = 1`. Loop terminates. First occurrence is at index `2`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log N)$	Executes two distinct binary searches. $O(\log N) + O(\log N) = O(2 \log N)$, which asymptotically simplifies to $O(\log N)$.
Space Complexity	$O(1)$	Strictly constant auxiliary memory footprint.

C++

```
#include <bits/stdc++.h>
using namespace std;

int firstOccurrence(vector<int>& arr, int size, int key) {
    int start = 0, end = size - 1;
    int ans = -1;

    while (start <= end) {
        int mid = start + (end - start) / 2;

        if (arr[mid] == key) {
            ans = mid;          // Record potential answer
            end = mid - 1;      // Force search leftwards
        }
        else if (key > arr[mid]) {
            start = mid + 1;
        }
        else if (key < arr[mid]) {
            end = mid - 1;
        }
    }
    return ans;
}

int lastOccurrence(vector<int>& arr, int size, int key) {
    int start = 0, end = size - 1;
    int ans = -1;

    while (start <= end) {
        int mid = start + (end - start) / 2;

        if (arr[mid] == key) {
            ans = mid;          // Record potential answer
```

```
            start = mid + 1;   // Force search rightwards
        }
        else if (key > arr[mid]) {
            start = mid + 1;
        }
        else if (key < arr[mid]) {
            end = mid - 1;
        }
    }
    return ans;
}
```

Problem 2: Peak Index in a Mountain Array (LeetCode 852)

1. Problem Statement & Constraints: An array forms a "mountain" if it strictly ascends to a peak element and then strictly descends. Find the exact array index of the peak element.

2. Core Intuition: The array consists of two distinct monotonic slopes: an ascending slope and a descending slope. We evaluate the mathematical relationship between `arr[mid]` and its immediate right neighbor `arr[mid+1]`.

- **Ascending Slope Condition:** If `arr[mid] < arr[mid+1]`, the slope is rising. The peak mathematically *must* exist to the right of `mid`.
- **Descending/Peak Condition:** If `arr[mid] > arr[mid+1]`, the slope is falling (or we are sitting exactly on the peak). The peak must exist at `mid` or to its left.

3. Algorithmic Steps:

1. Initialize `start = 0` and `end = arr.size() - 1`.
2. Open a `while (start < end)` loop. *(Crucial trick: Do NOT use `<=` because we are modifying `end = mid` and `start == end` will trigger an infinite loop).*
3. Calculate `mid = start + (end - start) / 2`.
4. Evaluate slope: `if (arr[mid] < arr[mid+1])`, update `start = mid + 1`.
5. Otherwise, update `end = mid`. *(Do not use `mid - 1` because `mid` itself could be the valid peak).*
6. When the loop halts, `start` and `end` will be pointing to the exact same index. Return `start`.

4. Dry Run:

Input: `arr = [1, 10, 5, 2, 0]`

1. `start = 0, end = 4`. `mid = 2` (value `5`). Check `5 < 2` (False). Descending slope. `end = mid = 2`.

2. `start = 0, end = 2`. `mid = 1` (value `10`). Check `10 < 5` (False). Descending slope/Peak. `end = mid = 1`.
3. `start = 0, end = 1`. `mid = 0` (value `1`). Check `1 < 10` (True). Ascending slope. `start = mid + 1 = 1`.
4. `start = 1, end = 1`. Loop condition `start < end` breaks. Return `1`. (Index 1 is value 10).

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log N)$	Discards half the mountain array on every iteration using slope evaluation.
Space Complexity	$O(1)$	No external data structures required.

C++

```
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {
        int start = 0;
        int end = arr.size() - 1;

        // Loop halts when pointers converge at the peak
        while (start < end) {
            int mid = start + (end - start) / 2;

            // Ascending slope check
            if (arr[mid] < arr[mid+1]) {
                start = mid + 1;
            }
            // Descending slope or currently at peak
            else {
                end = mid;
            }
        }
        return start;
    }
};
```

Problem 3: Find Pivot Index (Prefix Sum Paradigm)

(Note: While not a Binary Search problem, this highlights mathematical array traversal using sliding accumulators)

1. Problem Statement & Constraints: Given an array of integers `nums`, calculate the pivot index. The pivot index is the index where the sum of

all elements strictly to the left equals the sum of all elements strictly to the right. Return `-1` if no such index exists.

2. Core Intuition: Instead of utilizing an $O(N^2)$ brute force approach (summing left and right arrays from scratch for every single index), we can pre-calculate the total sum of the entire array. As we iterate left to right, we dynamically subtract the current element from our `rsum` (Right Sum) and check if it matches our `lsum` (Left Sum). If it doesn't match, we add the element to `lsum` and proceed.

3. Algorithmic Steps:

1. Initialize `lsum = 0` and `rsum = 0`.
2. Iterate through `nums` and accumulate the total into `rsum`.
3. Open a `for` loop `j = 0` to `nums.size() - 1`.
4. Deduct the current value from the right side: `rsum -= nums[j]`.
5. Evaluate equilibrium: `if (lsum == rsum)`, return `j` immediately.
6. If False, transition the value to the left side: `lsum += nums[j]`.
7. Return `-1` if loop finishes.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	Requires exactly two independent, sequential passes over the array. $O(N) + O(N) = O(N)$.
Space Complexity	$O(1)$	Memory is statically bound to `lsum` and `rsum` trackers.

C++

```
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int lsum = 0;
        int rsum = 0;

        // Pre-compute the absolute total sum of the array
        for (auto i : nums) {
            rsum += i;
        }

        for (int j = 0; j < nums.size(); j++) {
            // Remove current element from right half
            rsum -= nums[j];

            // Evaluate equilibrium
            if (lsum == rsum) {
                return j;
```

```
            }

            // Append current element to left half for next iteration
            lsum += nums[j];
        }

        return -1;
    }
};
```